

올바른 테스트 오라클 생성

: 뮤테이션 전파로 관찰 대상을 식별

정지나, 김운호

한양대학교 소프트웨어공학연구실



배경: 테스트 오라클의 중요성 및 어려움

- $Test = Prefix(Execute) + Oracle(Judgement)$
- ❖ 코드를 실행하는 **입력 생성**은 다양한 방식으로 자동화 생성에 **성공**.
- ❖ 입력에 대해서 관측이 올바른지 판단하는 **오라클 생성**은 여전히 난제.

배경: 기존 연구의 한계

- **회귀 검증 환경에 제한된 오라클(EvoSuite/Randooop/PANTA)**
 - 현재 버전의 동작을 정답으로 고정 → 회귀 검증 환경에서 제한된 적용
- **낮은 결함 탐지, 높은 거짓 양성 문제(LLM 기반, TOGA/TOGLL^[1])**
 - TOGA (ICSE'22) : 8% 결함 탐지, 45% 거짓 양성
 - TOGLL (ICSE'25) : 17% 결함 탐지, 42% 거짓 양성

[1] S. B. Hossain and M. B. Dwyer, "TOGLL: Correct and Strong Test Oracle Generation with LLMs," in Proc. ICSE, 2025

낮은 결함 탐지와 거짓 양성의 위험성

- **FP(False Positive): 정상 코드에서 “관찰값 $\neq$ 기댓값” 으로 Fail 발생.**
 - 원인 : 코드와 문서를 근거로 LLM이 기댓값을 추론 $\rightarrow$ 실제 동작과 불일치.

- **결과적 위험성**

- ① 인간 검증 비용의 부활: 자동 오라클 생성의 의미 퇴화.
- ② 잘못된 수정 유발: 틀린 오라클을 통과 시키기 위해, 정상 코드를 잘못 수정.

>> “양치기 소년”의 효과로 **오라클의 신뢰 붕괴!**

결함을 잘 탐지하기 위한 필요 조건^[2,3,4]

- | | |
|------------------------|-----------------------------|
| ■ Reachability | 도달 : 결함이 있는 코드가 실행. |
| ■ Infection | 오염 : 상태가 잘못된 값으로 변경. |
| ■ Propagation | 전파 : 오염된 상태가 관찰 가능한 출력에 반영. |
| ■ Revealability | 노출 : 오라클로 출력을 실제로 관찰 · 검사. |

>> 기존 LLM 기반 오라클 추론은 “관찰 대상의 정보”를 반영하지 않음.

>> 낮은 결함 탐지율이라는 결과.

[2] Jeffrey M. Voas, "PIE: A Dynamic Failure-Based Technique," IEEE TSE, vol. 18, no. 8, pp. 717–727, 1992.

[3] L. J. Morell, "A Theory of Fault-Based Testing," IEEE TSE, vol. 16, no. 8, pp. 844–857, 1990.

[4] P. Ammann and J. Offutt, Introduction to Software Testing, 2nd ed., Cambridge Univ. Press, 2016. (RIPR model)

Example: Commons-Math-3.6.1

/*addValue 메서드를 사용하여 추가된 데이터 값 스트림에 대한 요약 통계를 계산합니다.*/

```
public void addValue(double value) {  
    sumImpl.increment(value);  
    sumsqImpl.increment(value);  
    minImpl.increment(value);  
    maxImpl.increment(value);  
    sumLogImpl.increment(value);  
    secondMoment.increment(value);  
    ...  
}
```

해당 항목 및 관찰 가능한 getter 목록	
합계	getSum()
제곱합	getSumsq()
최솟값	getMin()
최댓값	getMax()
로그합	getGeometricMean()
2차모멘트	getVariance()/getStandardDeviation()

▪ 문서와 메서드를 근거로 LLM이 추론한 오라클 → 모호한 문서 정보로 인하여, 대표적인 요약 통계 위주 검사

- ① assertEquals(expect, s.getSum()); // 합계
- ② assertEquals(expect, s.getMin()); // 최솟값
- ③ assertEquals(expect, s.getMax()); // 최댓값

>> 모두 “문서에 충실한” 오라클을 생성하지만, 명확한 “관찰 대상의 정보”의 부재로 모든 값을 검증하지 않음.

KEY IDEA: With Propagation Information

/*addValue 메서드를 사용하여 추가된 데이터 값 스트림에 대한 요약 통계를 계산합니다.*/

```
public void addValue(double value) {  
    sumImpl.increment(value);  
    sumsqImpl.increment(value);  
    minImpl.increment(value);  
    maxImpl.increment(value);  
    ★ M1 sumLogImpl.increment(value * 2);  
    secondMoment.increment(value);  
    ...  
}
```

해당 항목 및 관찰 가능한 getter 목록	
합계	getSum()
제곱합	getSumsq()
최솟값	getMin()
최댓값	getMax()
로그합	getGeometricMean()
2차모멘트	getVariance()/getStandardDeviation()

- 뮤테이션 테스트를 통한 전파된 관찰 대상을 식별 → 해당 관찰 대상을 포함하여 LLM 오라클 추론

⑤ assertEquals(expect, s.getGeometricMean()); // 기하평균

>> “문서에 충실한” 오라클에 더불어, 명확한 “관찰 대상의 정보”로 결함이 노출될 수 있는 오라클 생성!

※ 뮤테이션이 도달→오염→전파를 성립시키고, 추가된 assert가 마지막 노출을 보장 → 결함 탐지 RIPR 조건 성립

※ Coupling Effect : 단순 뮤테이션에 민감한 지점은 실제의 복잡한 결함에도 민감함.^[5,6,7]

[5] DeMillo, Lipton, Sayward (1978), "Hints on Test Data Selection," IEEE Computer.

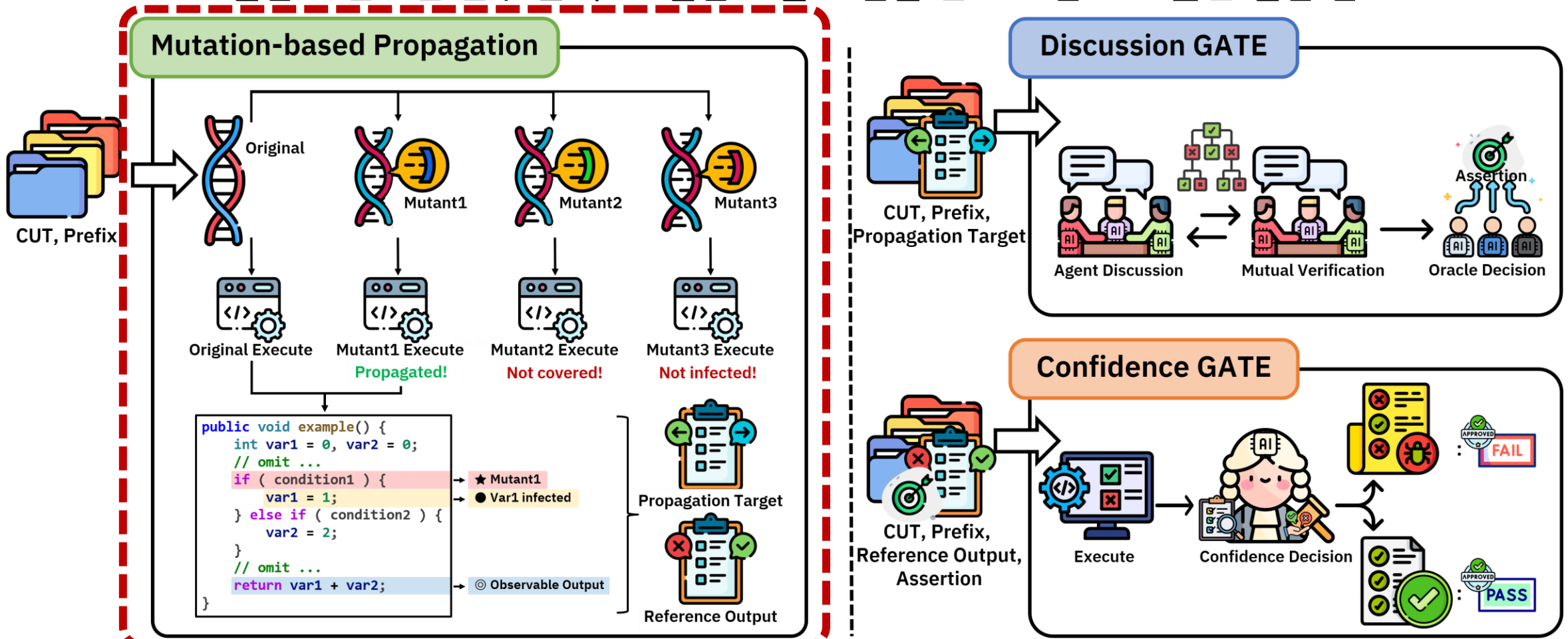
[6] Andrews, Briand, Labiche (2005), "Is Mutation an Appropriate Tool for Testing Experiments?," ICSE

[7] Just, Jalali, Inozemtseva, Ernst, Holmes, Fraser (2014), "Are Mutants a Valid Substitute for Real Faults in Software Testing?," FSE

Design: Propagation-guided Oracle Generation

WHERE : 검사 위치 – Propagation Analysis

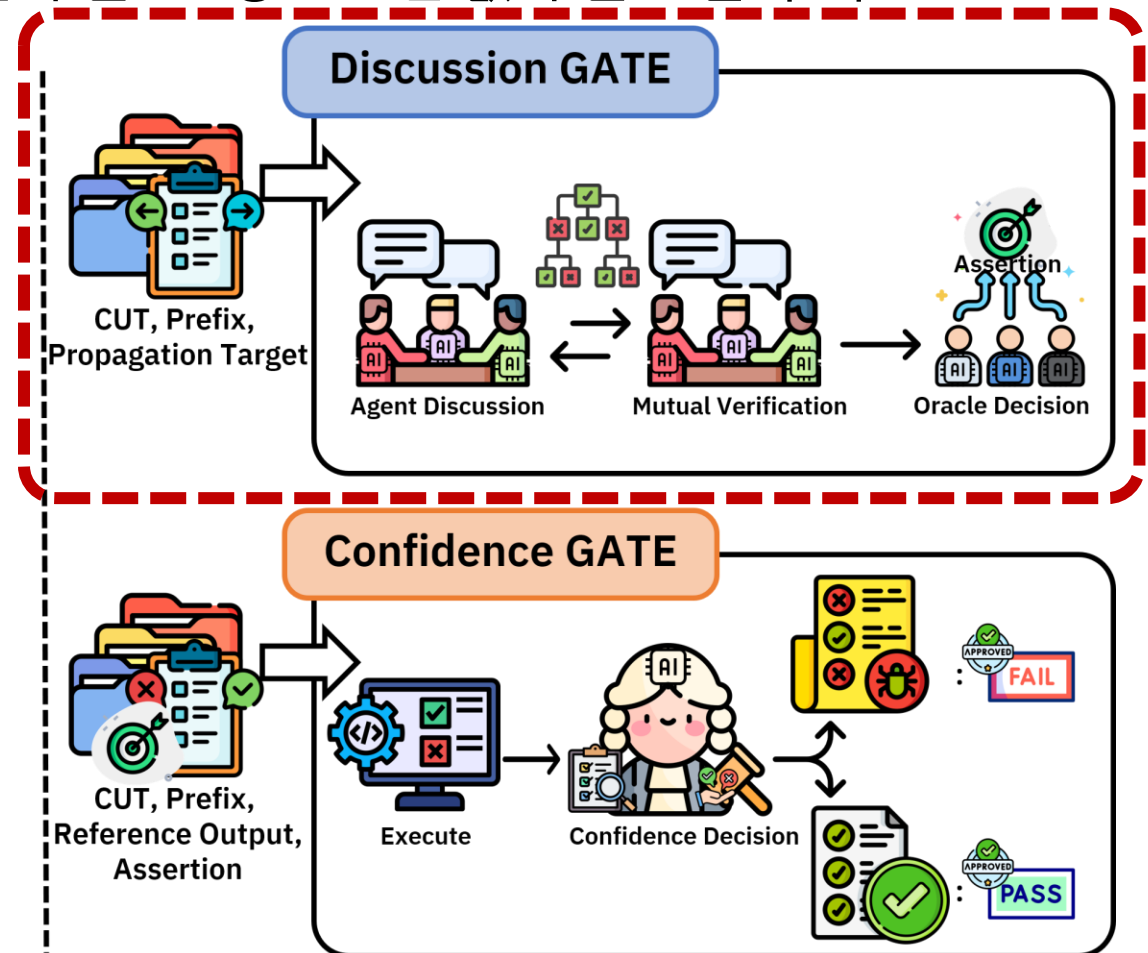
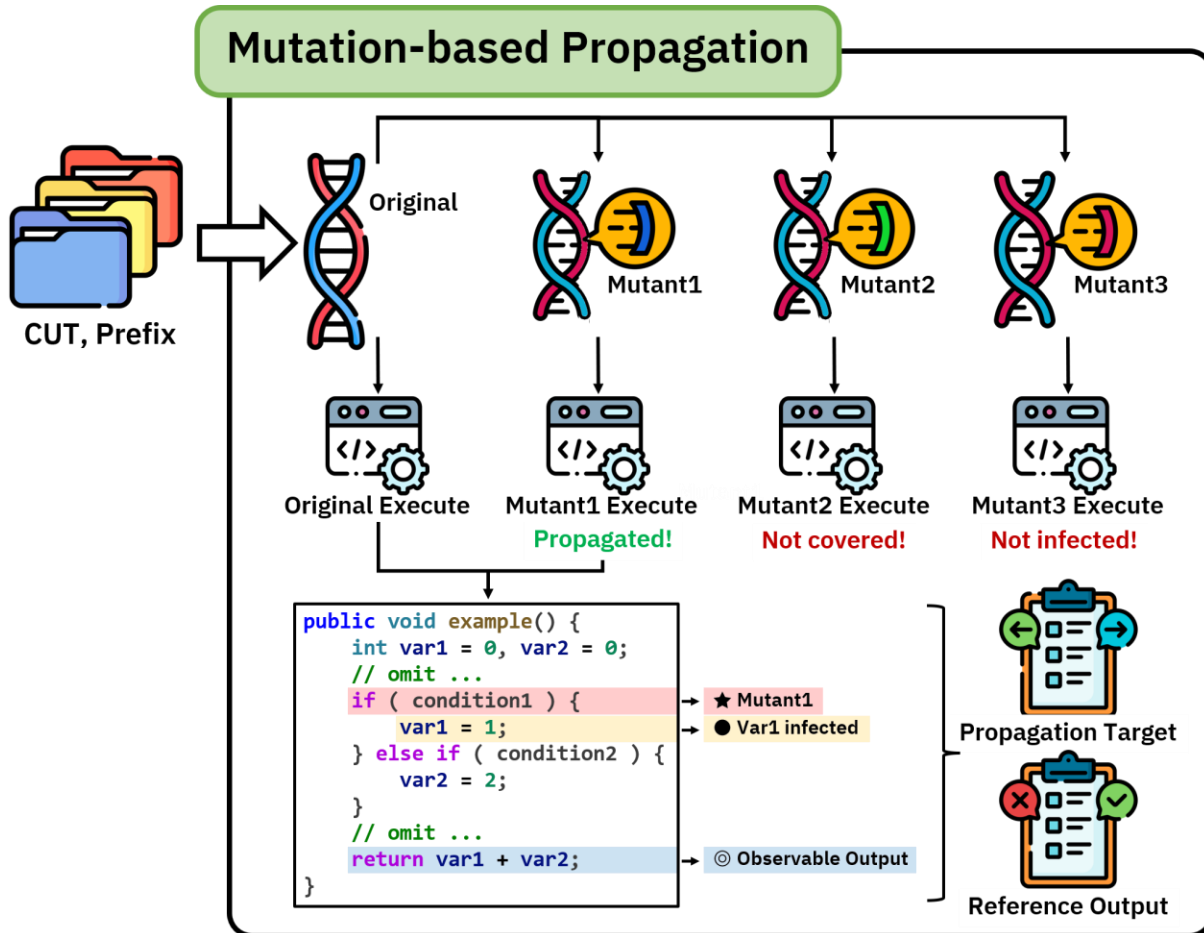
- 결합이 전파된 변수 분석 → 결합 노출 지점을 반드시 검사 → 높은 결함 탐지



Design: Propagation-guided Oracle Generation

WHAT : 기대 값 생성 – Discussion GATE

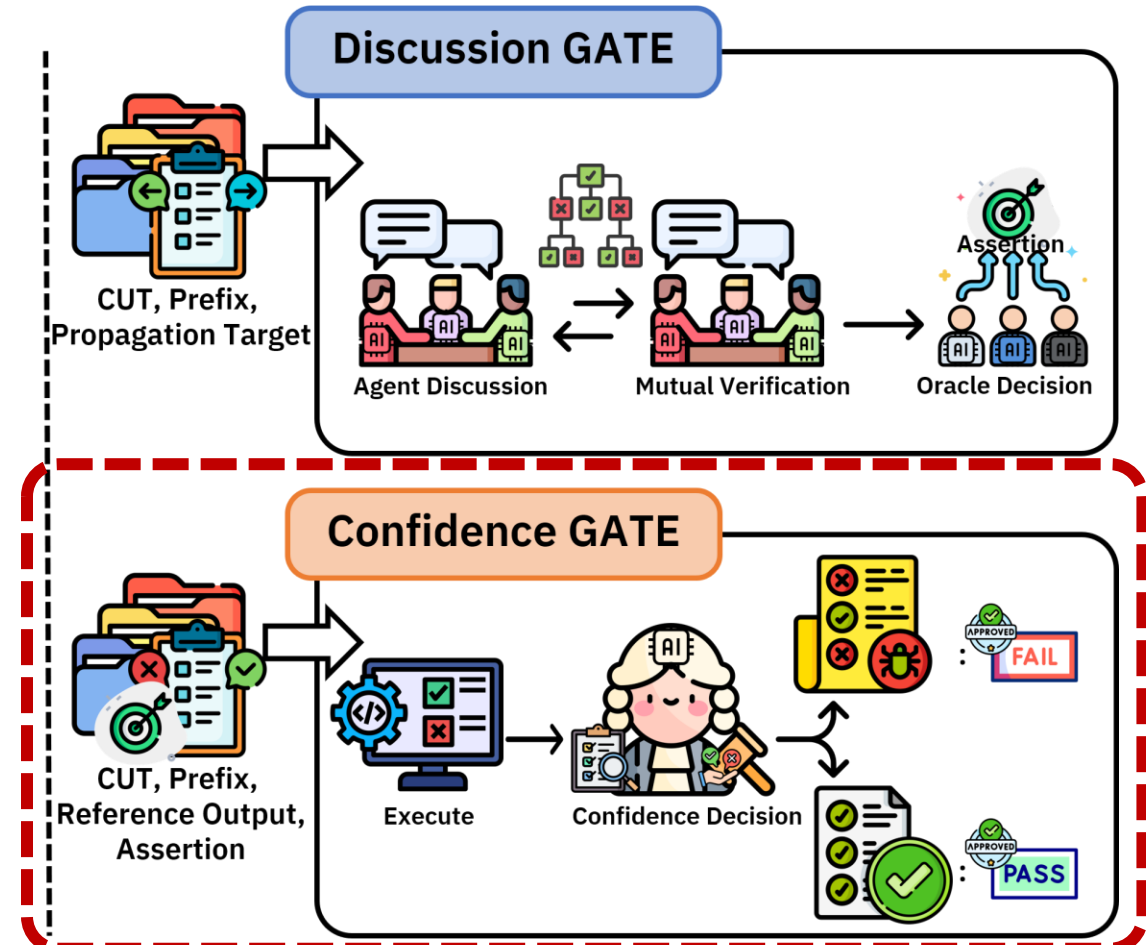
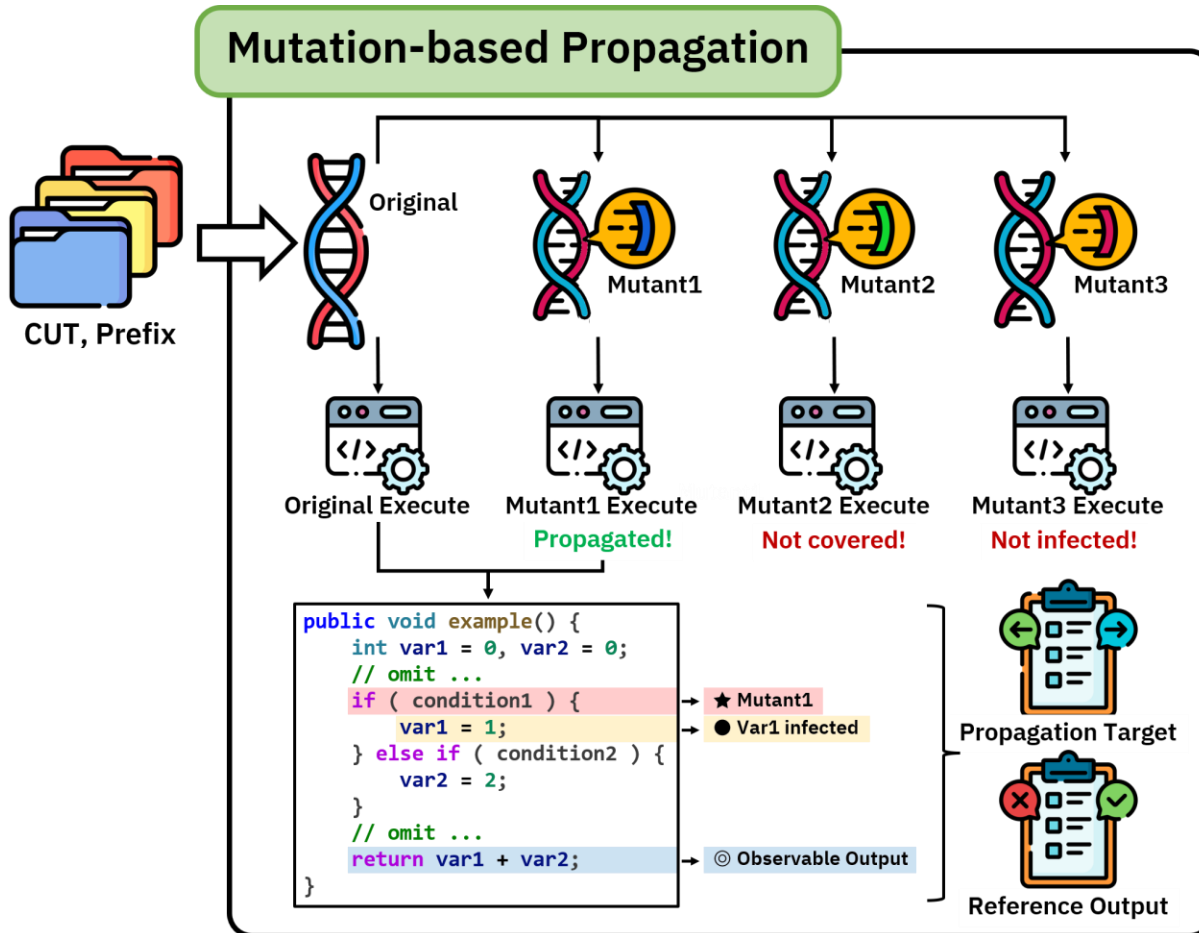
- 토너먼트 식 LLM 에이전트 토의를 통한 추론 → 명세 기반 값 추론 + 환각 억제



Design: Propagation-guided Oracle Generation

■ VERIFY : 오라클 검증 – Confidence GATE

- 생성된 Assert 를 실행으로 신뢰도 판정 → 신뢰 할 수 있는 오라클 출력



POSTER

Email: gjung@hanyang.ac.kr

▼ 메일 보내기 바로가기



관련 질문, 조언, 논의, 응원, 그냥 연락 모두 환영 합니다!
많은 관심 기다리겠습니다.
- 감사합니다 -



뮤테이션 전파 분석을 활용한 LLM 기반 테스트 오라클 생성

정지나, 김윤호
한양대학교 소프트웨어공학연구소

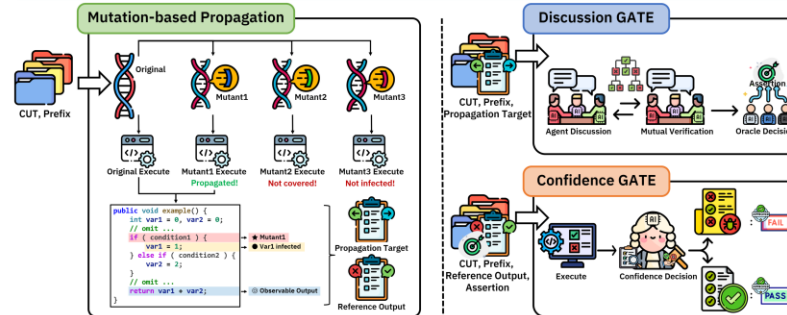
배경 및 기존 연구

- 테스트 오라클의 중요성: $Test = Prefix(Execute) + Oracle(Judgement)$
 - 입력(Prefix)은 SBST-LLM으로 자동화되었지만, 관측이 옳은지 판단하는 오라클(Judgement) 생성은 여전히 난제.
- 이 난제에 대한 기존 접근의 두 실패: 회귀 편향(Regression bias) · 낮은 탐지 & 거짓 양성(FP)
 - (A) 회귀 오라클 연구: 현재 버전의 동작을 정답으로 고정 → 회귀 검증 환경에서 한정된 적용.
 - (B) LLM 기반: 17%의 낮은 결함 탐지 + 오라클 신뢰를 붕괴시키는 45%의 높은 거짓 양성^[1]
 - 원인: 기존 오라클 생성은 "여기를 검사해야 결함이 드러나는가"에 대한 추론 근거 부족 → 결함률 높임 · 정상 코드 실패 → 낮은 결함 탐지 · 높은 거짓 양성.

동기 및 접근 방식

- 결함 탐지를 잘 하기 위한 필요 조건 - RPIR 모델: 결함은 도달(R)→오염(I)→전파(P)→노출(R)이 모두 성립해야 드러남^[2]
 - 도달·오염·전파: 결함이 어느 관찰 대상까지 어떻게 전파되는지는 문서·코드 정보로는 알 수 없음 → Mutation Testing으로 실측해야 함.
 - Mutation Testing으로 전파 정보를 실측 → "여기를 검사해야 하는가(관찰 대상)"를 오라클에 공급 → 올바른 기대 값 추론 → 결함 탐지
- LLM 에이전트에 전파 정보를 공급 → 관찰 대상을 확정 → 토의를 통한 기대 값 추론 → 실행 결과로 검증 → 올바른 Test Oracle 생성!
 - Mutation-based Propagation (WHERE, 관찰 대상 확정): Mutation Testing → 결함이 드러나는 관찰 대상을 실측
 - ※ 전제(Fault coupling): 단순 유턴트에 민감한 지점은 실제의 복잡한 결함에도 민감함.
 - Discussion GATE (WHAT, 기대 값 추론): 조건의 위치 · 관찰 대상만 제공^[3] → 다중 에이전트 토의로 기대 값 추론
 - Confidence GATE (VERIFY, 오라클 검증): 생성된 Assertion을 실제 실행 값과 대조 · 검증 → 신뢰가능한 Pass/Fail

Propagation-guided Oracle



- Mutation-based Propagation Analysis (WHERE)
 - 원본 vs. 유턴트 실행 > 차이를 관찰 > 결함이 전파된 유턴트만 유요신호로 판별 > 전파된 타겟(조건, 변수, 관찰 출력)을 식별
 - 산출물은 값 공개 범위에 따라 2가지로 분류(Propagation Target / Reference Output) > 버그 탐지 최대화, 거짓 양성 최소화 목적.
- Discussion GATE (WHAT): Tournament Style Agent Discussion & Mutual Verification
 - 입력: Propagation Target - 관찰 대상과 분기가 의존하는 조건이 담긴 (구체적인 Mutant 값은 비공개, 회귀 편향 방지)
 - 토너먼트 방식 다중 에이전트 토의: 각 에이전트가 Assertion 후보 제출 > 상호 피드백 · 검증 > 의견 유지 · 변경 > 합의로 추론된 기대 값 결정.
- Confidence GATE (VERIFY): Final Decision
 - 입력: Reference Output - 구체적인 값을 포함한 전파 분석 결과와 에이전트 토론을 통해서 결정된 Assert의 실행 결과
 - 실제 실행 결과와 대조 · 검증 > 신뢰 가능한 Pass / 신뢰 가능한 Fail 판정 > 거짓 양성 제어

실험 계획

- 전파 정보를 LLM에게 제공하는 것이 버그 검출 능력을 향상 하는데 도움이 되는지 실증적 평가.
- 실제 실행 결과를 LLM 에이전트가 피드백 하는 것이 회귀 편향, 거짓 양성, 버그 검출 능력과의 상관관계 평가.

[1] S. B. Hossain and M. B. Dwyer, "TOGL: Correct and Strong Test Oracle Generation with LLMs," in Proc. ICSE, 20025

[2] P. Ammann and J. Offutt, Introduction to Software Testing, 2nd ed, Cambridge Univ. Press, 2016. (RPIR model)

[3] Just, Jalali, Inozemtseva, Ernst, Holmes, Fraser (2014), "Are Mutants a Valid Substitute for Real Faults in Software Testing?", FSE